Assignment- 4

- Aim: Bucket Sort for student percentage Analysis
- Problem Statement: Write a C program to store FY percentage of students in an array. Write a function for sorting array of floating-point numbers in ascending order using Bucket sort & Display top five scores.
- Objective:
  1. Understand Bucket Sort
  2. Implement Bucket sort using functions.
  3. Sort floating-point percentages
  4. Display top five scores
  5. Analyze complexity.
- FAQs:

1. What is Bucket Sort?

A- A sorting algorithm that distributes elements into several Buckets, sort each bucket individually then concates them.

2. Why to use it for floating-point data?

A- It works well when data is uniformly distributed over a range (0-100) & maps naturally onto buckets. This avoids repeated comparisions accross the whole dataset & speeds up sorting compared to general purpose algorithms.

3. What is a bucket?

A- A bucket is a temporary list/array that stores all elements falling within a specific value range.

- Once filled, each bucket is sorted independently before being merged back together.

4. What is worst-case complexity?

A- The worst case is $O(n^2)$, which happens when all elements are pushed into a single bucket, effectively degrading to insertion sort on the entire array.

- This occurs when data is not uniformely distributed.

5. What are its applications?

A- Bucket sort is used in grading & ranking systems, histogram generation, load balancing across servers, and any scenario involving uniformly distributed numeric data like scores or measurements.

- Implementation:
  - Platform:
    - 64-bit open source Linux
    - Open source C programming tool

- Algorithm:

1. Start
2. Read 'n' & input the array of student % arr[0...n]
3. Create 'n' empty buckets, each able to hold multiple values.
4. For each i from 0 to n-1:
   - Compute index = (arr[i] * n) / 100
   - Insert arr[i] into bucket[index]
5. Sort each bucket individually using insertion sort
6. Concate all buckets in order to get the fully sorted array
7. STOP.

- Pseudo Code:

```
BucketSort (arr[], n):
    1. Create n empty buckets.
    2. For each element arr[i]:
            index = n*arr[i]/100
            insert arr[i] into bucket
    3. Sort each bucket individually
    4. Concate all bucket in order → sorted array
    5. Extract & display the last 5 elements.
```

- Time Complexity:

Case	Complexity
Best	$O(n+k)$
Average	$O(n+k)$
Worst	$O(n^2)$
Space.	$O(n+k)$

- Conclusion:
  - Bucket sort efficiently sorts uniformly distributed floating point % & helps identify the top 5 performers.

C Program:

```
#include <stdio.h>

void bucketSort(float a[], int n)
{
    float bucket[100][100];
    int count[100] = {0};
    int i, j, k;
    float temp;

    for (i = 0; i < n; i++)
    {
        k = (int)a[i];
        bucket[k][count[k]] = a[i];
        count[k]++;
    }

    for (i = 0; i < 100; i++)
    {
        for (j = 0; j < count[i] - 1; j++)
        {
            int l;

            for (l = 0; l < count[i] - j - 1; l++)
            {
                if (bucket[i][l] > bucket[i][l + 1])
                {
                    temp = bucket[i][l];
                    bucket[i][l] = bucket[i][l + 1];
                    bucket[i][l + 1] = temp;
                }
            }
        }
    }

    k = 0;

    for (i = 0; i < 100; i++)
    {
        for (j = 0; j < count[i]; j++)
        {
            a[k] = bucket[i][j];
            k++;
        }
    }
}

void displayTopFive(float a[], int n)
{
    int i, limit;

    if (n < 5)
        limit = n;
    else
        limit = 5;

    printf("\nTop %d Scores:\n", limit);

    for (i = n - 1; i >= n - limit; i--)
    {
        printf("%.2f\n", a[i]);
    }
}

int main()
{
    float percentage[100];
    int n, i;

    printf("Enter number of students: ");
    scanf("%d", &n);

    printf("Enter percentages of students:\n");

    for (i = 0; i < n; i++)
    {
        scanf("%f", &percentage[i]);
```

```
    }

    bucketSort(percentage, n);

    printf("\nPercentages in Ascending Order:\n");

    for (i = 0; i < n; i++)
    {
        printf("%.2f ", percentage[i]);
    }

    displayTopFive(percentage, n);

    return 0;
}
```

Output:

```
Enter number of students: 10
Enter percentages of students:
91 92 91 83 82 84 95 82 88 73

Percentages in Ascending Order:
73.00 82.00 82.00 83.00 84.00 88.00 91.00 91.00 92.00 95.00

Top 5 Scores:
95.00
92.00
91.00
91.00
88.00
```